

Ford-Johnson (Merge-Insertion Sort) — comprendre l'algorithme avant de coder

1. Pourquoi cet algorithme est particulier

Un tri par fusion classique fait $O(n \log n)$ comparaisons, sans chercher à minimiser leur nombre exact. Ford-Johnson (1959) fait mieux : c'est l'algorithme qui utilise le **nombre de comparaisons minimal théorique** connu pour trier par comparaisons, pour beaucoup de valeurs de n . C'est pour ça qu'il combine trois idées à la fois :

Regrouper les éléments par paires et ne comparer qu'à l'intérieur de chaque paire.

Trier récursivement seulement la moitié des éléments (les plus grands de chaque paire).

Insérer le reste dans un ordre précis (pas au hasard !) pour que chaque insertion binaire ait la plus grande marge de recherche possible.

C'est ce point 3 — l'ordre d'insertion — qui rend l'algorithme déroutant au premier abord. Le reste (paires, récursion, insertion binaire) est standard.

2. Vue d'ensemble en 5 étapes

Pour trier une séquence de n éléments :

Former des paires : $\lfloor n/2 \rfloor$ paires, plus 1 élément seul si n est impair.

Comparer chaque paire : dans chaque paire, identifier le plus grand (a) et le plus petit (b). (1 comparaison par paire.)

Trier récursivement les a (les plus grands de chaque paire) avec le même algorithme — c'est ici que la récursion entre en jeu, et qu'on ne trie qu'*la moitié* des éléments.

Construire la "chaîne principale" de départ : $b_1, a_1, a_2, \dots, a_k$ (où $a_1 < a_2 < \dots < a_k$ viennent de l'étape 3, et b_1 est le plus petit partenaire, déjà connu $< a_1$).

Insérer les b restants (et l'élément impair s'il y en a un) dans la chaîne, **dans un ordre précis** basé sur les nombres de Jacobsthal, chacun via une **recherche binaire**.

3. Étape par étape avec un exemple concret

Prenons 11 éléments (volontairement un nombre pas rond, pour bien voir l'élément impair et un ordre d'insertion sur plusieurs "blocs") :



Séquence de départ : 7 2 9 4 1 8 3 6 10 5 11

3.1 Former les paires

On regroupe consécutivement (l'ordre de formation des paires n'a pas d'importance pour la correction, seulement pour l'implémentation) :



(7,2) (9,4) (1,8) (3,6) (10,5) -- 11 est impair, il reste seul de côté

3.2 Comparer chaque paire → séparer en `a` (grand) et `b` (petit)

Paire plus grand (a) plus petit (b)

(7,2) a=7 b=2

(9,4) a=9 b=4

(1,8) a=8 b=1

(3,6) a=6 b=3

(10,5) a=10 b=5

5 comparaisons faites ici, une par paire. Élément seul restant : `11`.

⚠ Point d'implémentation important : **chaque `b_i` reste "attaché" à son `a_i`** tout au long de l'algorithme (même quand on trie les `a` récursivement, `b_i` "suit" son partenaire). En pratique on garde une structure du type `std::pair<int,int>` (ou deux conteneurs parallèles), pas juste une liste de `int`.

3.3 Trier récursivement les `a` : `[7, 9, 8, 6, 10]`

On applique **le même algorithme** sur `[7, 9, 8, 6, 10]` (5 éléments). Sans détailler toute la récursion (elle suit exactement les mêmes étapes en plus petit), on obtient :



a triés : 6 < 7 < 8 < 9 < 10

Comme les `b` sont attachés à leur `a`, on garde la correspondance :



a1=6 (b1=3) a2=7 (b2=2) a3=8 (b3=1) a4=9 (b4=4) a5=10 (b5=5)

3.4 Construire la chaîne principale de départ

On sait déjà que `b1 < a1` (c'est comme ça qu'on les a définis à l'étape 3.2). Donc on peut démarrer la chaîne avec :



Chaîne principale : b1, a1, a2, a3, a4, a5
= 3, 6, 7, 8, 9, 10

Cette chaîne de 6 éléments est **déjà triée** — aucune comparaison de plus n'a été nécessaire pour l'obtenir, on a juste assemblé des faits déjà connus.

Reste à insérer : $b_2=2$, $b_3=1$, $b_4=4$, $b_5=5$, et l'élément impair 11 .

3.5 L'ordre d'insertion : les nombres de Jacobsthal

C'est le cœur de l'astuce. Si on insérait b_2 , b_3 , b_4 , b_5 dans l'ordre naturel, certaines insertions binaires auraient une plage de recherche inutilement petite. Ford et Johnson ont montré que l'ordre optimal suit la suite de **Jacobsthal** :


 $J(0)=0, J(1)=1, J(n) = J(n-1) + 2*J(n-2)$
Suite : 0, 1, 1, 3, 5, 11, 21, 43, 85, ...

Règle d'insertion : on insère les b par blocs, en descendant : pour chaque k , on insère $b[J(k)]$, $b[J(k)-1]$, ..., $b[J(k-1)+1]$ (dans cet ordre décroissant d'indice), puis on passe au bloc suivant.

Concrètement, l'ordre global d'insertion des indices b_i est toujours :


 $b_3, b_2, b_5, b_4, b_{11}, b_{10}, b_9, b_8, b_7, b_6, b_{21}, b_{20}, \dots, b_{12}, \dots$

(le premier bloc va de $J(3)=3$ à $J(2)+1=2 \rightarrow b_3, b_2$; le suivant de $J(4)=5$ à $J(3)+1=4 \rightarrow b_5, b_4$; le suivant de $J(5)=11$ à $J(4)+1=6 \rightarrow b_{11} \dots b_6$; etc.)

Dans notre exemple, on n'a que b_2, b_3, b_4, b_5 à insérer (pas de b_6 et au-delà). L'ordre devient donc :


 b_3, b_2, b_5, b_4

L'élément impair restant (11) est traditionnellement inséré **en dernier dans son bloc naturel** (il est traité comme un "b" supplémentaire dont l'indice suit immédiatement le dernier b réel — ici après b_5 , donc dans le même bloc que b_5, b_4 , mais après eux puisqu'il n'a pas de contrainte d'ordre avec les a).

3.6 Insérer un par un, par recherche binaire

Chaîne actuelle : $[3, 6, 7, 8, 9, 10]$

Insérer $b_3=1$ — recherche binaire *seulement dans la portion de la chaîne antérieure à $a_3=8$* (règle clé : b_i ne peut jamais se trouver après a_i , donc on limite la recherche à $[\dots, a_i[$ pour économiser des comparaisons) :


Portion valide : $[3, 6, 7] \rightarrow 1$ s'insère avant 3
Chaîne : $[1, 3, 6, 7, 8, 9, 10]$

Insérer $b_2=2$ — limite de recherche : avant $a_2=7$:



Portion valide : [1, 3, 6] → 2 s'insère entre 1 et 3

Chaîne : [1, 2, 3, 6, 7, 8, 9, 10]

Insérer **b5=5** — limite : avant **a5=10** :



Portion valide : [1, 2, 3, 6, 7, 8, 9] → 5 s'insère entre 3 et 6

Chaîne : [1, 2, 3, 5, 6, 7, 8, 9, 10]

Insérer **b4=4** — limite : avant **a4=9** :



Portion valide : [1, 2, 3, 5, 6, 7, 8] → 4 s'insère entre 3 et 5

Chaîne : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Insérer l'élément impair **11** — pas de contrainte de borne supérieure (aucun **a** associé) : recherche binaire sur toute la chaîne :



Chaîne : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

Résultat final, trié.

4. Pourquoi limiter la recherche binaire à **[..., ai[** ?

C'est l'optimisation qui économise des comparaisons. Comme **bi < ai** a été établi dès l'étape 3.2 (comparaison de la paire), et que **ai** est déjà à sa place définitive dans la chaîne, on **sait déjà** que **bi** ne peut pas se trouver après **ai**. Chercher au-delà serait donc inutile — on borne la recherche binaire à la sous-séquence qui précède **ai** dans la chaîne courante.

C'est exactement pour ça que l'ordre de Jacobsthal est choisi : il maximise la taille de cette "portion valide" à chaque étape (en insérant d'abord les **b** dont le **a** associé est déjà loin dans la chaîne), ce qui minimise le nombre moyen de comparaisons de recherche binaire sur l'ensemble du tri.

5. Cas de base de la récursion

n == 0 ou **n == 1** → déjà trié, rien à faire.

Sinon, on répète les 5 étapes ci-dessus sur les **a** (récursion), jusqu'à tomber sur un cas de base.

6. Points d'attention pour l'implémentation en C++98 / STL

Garder l'association `(a, b)` : utiliser `std::pair<int, int>` (ou une structure équivalente) plutôt que deux listes séparées qu'on risque de désynchroniser.

Générer les nombres de Jacobsthal à la volée (une simple boucle avec la relation $J(n) = J(n-1) + 2 \cdot J(n-2)$), pas besoin de les coder en dur.

La borne supérieure de recherche `(ai)` doit être calculée dynamiquement : après plusieurs insertions, la position de `ai` dans la chaîne a pu changer (des éléments ont été insérés avant lui) — il faut retrouver sa position courante, pas supposer une position fixe.

L'élément impair (`straggler`) : à traiter comme un cas particulier — pas de `a` associé, donc pas de borne supérieure de recherche, sa recherche binaire couvre toute la chaîne.

Deux conteneurs différents demandés par le sujet (ex: `std::vector` et `std::list`/`std::deque`) : le sujet insiste pour **implémenter l'algorithme séparément pour chaque conteneur** plutôt qu'une fonction générique — donc prévoir deux implémentations distinctes (les opérations d'insertion/accès aléatoire n'ont pas le même coût selon le conteneur, ce qui est justement l'intérêt pédagogique de comparer les deux avec les mesures de temps demandées).

La recherche binaire : rien n'empêche d'utiliser `std::upper_bound`/`std::lower_bound` de `<algorithm>` (autorisé en Module 09), tant que l'algorithme de tri lui-même (la logique Ford-Johnson) est écrit par toi et pas juste `std::sort`.

7. Ce qu'il te reste à faire

Avec ce cours, tu as la logique complète. Les étapes concrètes pour coder `PmergeMe` :

Écrire une fonction qui génère les paires et sépare `a`/`b`.

Écrire la fonction récursive (appel sur les `a`).

Écrire la génération de la suite de Jacobsthal et l'ordre d'insertion qui en découle.

Écrire l'insertion binaire bornée par la position courante de `ai`.

Gérer le cas de l'élément impair.

Dupliquer proprement pour ton deuxième conteneur.

Chronométrer chaque implémentation séparément (voir l'exemple d'affichage du sujet).

N'hésite pas à me montrer ton code au fur et à mesure — on pourra tracer l'exécution comme on l'a fait pour `BitcoinExchange`.